ARQUITETURA TÉCNICA

Pipeline Automatizado de Análise de Performance Orgânica

Redes Sociais + Blog + SEO → Claude API → Relatório Diário/Semanal

Março 2026 | Documento para equipe de desenvolvimento

1. Visão Geral do Sistema

Este documento descreve a arquitetura de um pipeline automatizado que coleta dados de performance orgânica das redes sociais (Instagram, Facebook, YouTube, TikTok), do blog (via Google Analytics 4) e de SEO (via Google Search Console), envia para a API do Claude para análise estratégica e entrega o relatório via e-mail, Slack ou WhatsApp.

1.1 Diferenças vs. Pipeline de Ads

O pipeline orgânico tem características distintas do pipeline de tráfego pago. Os dados mudam de forma mais lenta (não há gasto diário flutuando), por isso a frequência recomendada é semanal para o relatório completo, com monitoramento diário apenas de alertas (posts viralizando, quedas bruscas de impressões, páginas desindexadas). Além disso, são mais fontes de dados distintas (5 a 6 APIs diferentes vs. 2 no pipeline de ads).

1.2 Fontes de Dados

Fonte	API	Métricas principais	Frequência
Instagram	Instagram Graph API	Alcance, impressões, engajamento, seguidores, saves	Diária
Facebook	Meta Graph API (Pages)	Alcance, engajamento, cliques no link	Diária
YouTube	YouTube Data API v3	Views, watch time, CTR thumb, inscritos, retenção	Diária
TikTok	TikTok Content Posting API / Research API	Views, likes, comments, shares, completion rate	Diária
Blog / Site	Google Analytics 4 Data API	Sessões, usuários, pageviews, tempo médio, bounce, conversões	Diária
SEO	Google Search Console API	Cliques, impressões, CTR, posição média, queries, páginas	Diária

1.3 Fluxo Resumido

CRON (segunda 07h) → Script Python → 6 APIs em paralelo → Consolida dados → Claude API (análise) → Relatório via E-mail/Slack/WhatsApp

2. Stack Tecnológica

Componente	Tecnologia	Observação
Linguagem	Python 3.11+	Mesmo do pipeline de ads
Instagram / Facebook	requests + Graph API	Mesmo token do Business Manager
YouTube	google-api-python-client	YouTube Data API v3 (cota: 10.000 unidades/dia)
TikTok	requests + TikTok API	Acesso limitado; alternativa: TikAPI
Google Analytics 4	google-analytics-data	GA4 Data API (não Universal Analytics)
Search Console	google-auth + searchconsole	Ou via google-api-python-client
IA / Análise	anthropic (Claude API)	claude-sonnet-4-20250514
Agendamento	CRON / GitHub Actions	Semanal (completo) + diário (alertas)

3. Coleta de Dados — Redes Sociais

3.1 Instagram (Graph API)

Pré-requisitos:

- Conta Instagram Business ou Creator conectada a uma Facebook Page
- Token de acesso de longa duração (System User via Business Manager)
- Permissões: instagram_basic, instagram_manage_insights, pages_read_engagement

Métricas disponíveis:

Nível	Endpoint	Métricas	Período
Conta	/{ig_user}/insights	impressions, reach, follower_count, profile_views	Últimos 30 dias
Post/Reel	/{media_id}/insights	impressions, reach, engagement, saved, shares, plays, comments	Lifetime
Stories	/{story_id}/insights	impressions, reach, exits, replies, taps_forward, taps_back	Lifetime (24h)

Código de referência:

```
import requests

BASE_URL = "https://graph.facebook.com/v19.0"
TOKEN = "SEU_ACCESS_TOKEN"
IG_USER_ID = "SEU_IG_BUSINESS_ACCOUNT_ID"

def coletar_insights_instagram():
    # 1. Métricas da conta (diário)
    url = f"{BASE_URL}/{IG_USER_ID}/insights"
    params = {
        "metric": "impressions,reach,follower_count,profile_views",
        "period": "day",
        "since": ontem_unix,
        "until": hoje_unix,
        "access_token": TOKEN,
    }
    conta = requests.get(url, params=params).json()

    # 2. Posts recentes
    media_url = f"{BASE_URL}/{IG_USER_ID}/media"
    media_params = {
        "fields": "id,caption,media_type,timestamp,permalink,"
```

```
                "like_count,comments_count",
        "limit": 25,
        "access_token": TOKEN,
    }
    posts = requests.get(media_url, params=media_params).json()

    # 3. Insights por post
    for post in posts["data"]:
        insights_url = f"{BASE_URL}/{post['id']}/insights"
        insights_params = {
            "metric": "impressions,reach,engagement,saved,shares",
            "access_token": TOKEN,
        }
        post["insights"] = requests.get(
            insights_url, params=insights_params).json()

    return {"conta": conta, "posts": posts}
```

3.2 Facebook Page (Graph API)

A coleta é similar ao Instagram, usando a mesma infraestrutura de tokens do Business Manager. O endpoint principal é /{page_id}/insights para métricas da página e /{page_id}/posts para métricas de posts individuais.

Código de referência:

```
PAGE_ID = "SEU_PAGE_ID"

def coletar_insights_facebook():
    # Métricas da página
    url = f"{BASE_URL}/{PAGE_ID}/insights"
    params = {
        "metric": "page_impressions,page_engaged_users,"
                  "page_fans,page_views_total,"
                  "page_actions_post_reactions_total",
        "period": "day",
        "since": ontem_unix,
        "until": hoje_unix,
        "access_token": TOKEN,
    }
    pagina = requests.get(url, params=params).json()

    # Posts recentes com métricas
    posts_url = f"{BASE_URL}/{PAGE_ID}/posts"
    posts_params = {
        "fields": "id,message,created_time,permalink_url,"
                  "shares,reactions.summary(true),"
                  "comments.summary(true)",
        "limit": 25,
        "access_token": TOKEN,
    }
    posts = requests.get(posts_url, params=posts_params).json()

    return {"pagina": pagina, "posts": posts}
```

3.3 YouTube (Data API v3 + Analytics API)

Pré-requisitos:

- Projeto no Google Cloud com YouTube Data API v3 e YouTube Analytics API habilitadas
- OAuth 2.0 credenciais (mesmo projeto do Google Ads, se já existe)
- Channel ID dos canais da Central de Concursos e Degrau Cultural

Métricas-chave:

API	Métricas	Uso
Data API v3	views, likes, comments, subscriberCount	Dados públicos de vídeos e canal
Analytics API	estimatedMinutesWatched, averageViewDuration, averageViewPercentage, subscribersGained	Dados privados: retenção, watch time, CTR

Código de referência:

```
from googleapiclient.discovery import build
from google.oauth2.credentials import Credentials

creds = Credentials.from_authorized_user_file("token.json")
youtube = build("youtube", "v3", credentials=creds)

def coletar_youtube():
    # Estatísticas do canal
    canal = youtube.channels().list(
        part="statistics,snippet",
        id=CHANNEL_ID
    ).execute()

    # Últimos vídeos publicados
    videos = youtube.search().list(
        part="id,snippet",
        channelId=CHANNEL_ID,
        order="date",
        maxResults=20,
        type="video"
    ).execute()

    # Métricas por vídeo
    video_ids = [v["id"]["videoId"] for v in videos["items"]]
    stats = youtube.videos().list(
        part="statistics,contentDetails",
        id=",".join(video_ids)
    ).execute()
```

```
    # YouTube Analytics - retenção e watch time
    yt_analytics = build("youtubeAnalytics", "v2", credentials=creds)
    analytics = yt_analytics.reports().query(
        ids="channel==MINE",
        startDate=ontem_str,
        endDate=ontem_str,
        metrics="views,estimatedMinutesWatched,averageViewDuration,"
                "averageViewPercentage,subscribersGained,likes",
        dimensions="video",
        sort="-views",
        maxResults=20
    ).execute()

    return {"canal": canal, "videos": stats, "analytics": analytics}
```

3.4 TikTok

O TikTok tem a API mais restritiva entre as plataformas. Existem três opções viáveis, cada uma com trade-offs.

Opção	Prós	Contras
TikTok Business API	Oficial, dados confiáveis	Requer aprovação, acesso limitado
TikAPI (terceiro)	Fácil integração, sem aprovação	Pago (~$30/mês), dependência externa
Export manual + CSV	Zero custo, zero dependência	Não automatizável

Código de referência (via TikAPI):

```
from tikapi import TikAPI

api = TikAPI("SUA_TIKAPI_KEY")

def coletar_tiktok():
    user = api.public.check(username="centralconcursos")
    user_info = user.json()["userInfo"]
    stats = user_info["stats"]

    videos = api.public.posts(secUid=user_info["user"]["secUid"])
    video_list = videos.json()["itemList"]

    resultados = []
    for v in video_list[:20]:
        resultados.append({
            "desc": v["desc"],
            "createTime": v["createTime"],
            "views": v["stats"]["playCount"],
            "likes": v["stats"]["diggCount"],
            "comments": v["stats"]["commentCount"],
            "shares": v["stats"]["shareCount"],
            "duration": v["video"]["duration"],
        })

    return {"perfil": stats, "videos": resultados}
```

Recomendação: Começar com export manual do TikTok Analytics (CSV) e integrar ao pipeline via leitura de arquivo. Migrar para TikAPI ou API oficial quando o volume justificar.

4. Coleta de Dados — Blog e SEO

4.1 Google Analytics 4 (GA4 Data API)

Pré-requisitos:

- Propriedade GA4 configurada no site/blog
- Service Account com permissão de Viewer na propriedade GA4
- Google Analytics Data API habilitada no Google Cloud

Código de referência:

```
from google.analytics.data_v1beta import BetaAnalyticsDataClient
from google.analytics.data_v1beta.types import (
    RunReportRequest, DateRange, Dimension, Metric, OrderBy
)

client = BetaAnalyticsDataClient()
GA4_PROPERTY_ID = "properties/SEU_PROPERTY_ID"

def coletar_ga4():
    request = RunReportRequest(
        property=GA4_PROPERTY_ID,
        date_ranges=[DateRange(start_date="7daysAgo", end_date="yesterday")],
        dimensions=[
            Dimension(name="pagePath"),
            Dimension(name="sessionDefaultChannelGroup"),
        ],
        metrics=[
            Metric(name="sessions"),
            Metric(name="totalUsers"),
            Metric(name="screenPageViews"),
            Metric(name="averageSessionDuration"),
            Metric(name="bounceRate"),
            Metric(name="conversions"),
            Metric(name="engagedSessions"),
        ],
        order_bys=[OrderBy(
            metric=OrderBy.MetricOrderBy(metric_name="screenPageViews"),
            desc=True
        )],
        limit=50,
    )
    response = client.run_report(request)

    paginas = []
    for row in response.rows:
        paginas.append({
            "pagina": row.dimension_values[0].value,
            "canal": row.dimension_values[1].value,
            "sessoes": row.metric_values[0].value,
            "usuarios": row.metric_values[1].value,
```

```
            "pageviews": row.metric_values[2].value,
            "duracao_media": row.metric_values[3].value,
            "bounce_rate": row.metric_values[4].value,
            "conversoes": row.metric_values[5].value,
        })

    return paginas
```

Segmentações recomendadas:

Para o blog de concursos, segmentar por: canal de aquisição (Organic Search vs Social vs Direct), dispositivo (mobile vs desktop — ~80% mobile), e páginas de entrada. Isso permite entender quais artigos trazem tráfego orgânico e quais convertem melhor.

4.2 Google Search Console API

Pré-requisitos:

- Propriedade verificada no Google Search Console
- Service Account com permissão na propriedade (ou OAuth2)
- Search Console API habilitada no Google Cloud

Importante: Os dados do Search Console têm um delay de 2-3 dias. O script deve buscar dados de 3 dias atrás para garantir completude.

Código de referência:

```
from googleapiclient.discovery import build
from google.oauth2 import service_account

SCOPES = ["https://www.googleapis.com/auth/webmasters.readonly"]
creds = service_account.Credentials.from_service_account_file(
    "service-account.json", scopes=SCOPES
)
service = build("searchconsole", "v1", credentials=creds)
SITE_URL = "https://www.centralconcursos.com.br"

def coletar_search_console():
    # 1. Performance por página
    req_paginas = {
        "startDate": tres_dias_atras,
        "endDate": ontem_str,
        "dimensions": ["page"],
        "rowLimit": 50,
        "orderBy": [{"fieldName": "clicks", "sortOrder": "DESCENDING"}],
    }
    paginas = service.searchanalytics().query(
        siteUrl=SITE_URL, body=req_paginas).execute()

    # 2. Performance por query
    req_queries = {
        "startDate": tres_dias_atras,
        "endDate": ontem_str,
        "dimensions": ["query"],
        "rowLimit": 100,
        "orderBy": [{"fieldName": "impressions", "sortOrder": "DESCENDING"}],
    }
    queries = service.searchanalytics().query(
        siteUrl=SITE_URL, body=req_queries).execute()

    # 3. Query + página (para oportunidades de SEO)
    req_oport = {
        "startDate": sete_dias_atras,
        "endDate": ontem_str,
        "dimensions": ["query", "page"],
        "dimensionFilterGroups": [{
            "filters": [{
                "dimension": "query",
```

```
                "operator": "contains",
                "expression": "concurso"
            }]
        }],
        "rowLimit": 200,
    }
    oportunidades = service.searchanalytics().query(
        siteUrl=SITE_URL, body=req_oport).execute()

    return {
        "paginas": paginas.get("rows", []),
        "queries": queries.get("rows", []),
        "oportunidades": oportunidades.get("rows", []),
    }
```

Análises de SEO que o Claude pode fazer:

- **Quick wins:** Queries com muitas impressões mas posição média 5-20 (perto da 1ª página)
- **CTR abaixo do esperado:** Páginas na posição 1-3 com CTR baixo (title/description precisam melhorar)
- **Canibalização:** Múltiplas páginas competindo pela mesma query
- **Oportunidades de conteúdo:** Queries com volume mas sem página dedicada
- **Páginas em queda:** Comparação semana vs semana anterior

5. Integração com Claude API

5.1 System Prompt — Redes Sociais

```
SYSTEM_PROMPT_SOCIAL = """
Você é um analista sênior de social media especializado em
marketing educacional brasileiro para concursos públicos.
Você analisa as contas das marcas Central de Concursos e Degrau Cultural.

Estrutura do relatório:

1. RESUMO EXECUTIVO
   - Visão geral cross-platform
   - Destaque da semana (melhor post/vídeo)
   - Crescimento de seguidores/inscritos

2. INSTAGRAM
   - Alcance e impressões (comparação semanal)
   - Top 3 posts por engajamento
   - Taxa de salvamento (indicador de conteúdo de valor)
   - Performance Reels vs Feed vs Stories

3. FACEBOOK
   - Alcance orgânico e engajamento
   - Melhor horário/formato

4. YOUTUBE
   - Watch time total e médio
   - CTR de thumbnail
   - Retenção média
   - Inscritos ganhos vs perdidos

5. TIKTOK
   - Views totais e média
   - Completion rate
   - Vídeos com melhor share rate

6. ALERTAS
   - Engajamento anômalo (positivo ou negativo)
   - Queda de alcance >20%
   - Vídeos com retenção <30% (hook fraco)

7. RECOMENDAÇÕES DE CONTEÚDO
   - O que replicar (formatos/temas que performaram)
   - O que evitar
   - Sugestões de temas baseadas nos dados

Regras:
- Nunca invente dados.
- Considere o nicho de concursos públicos.
```

```
- Recomendações devem ser acionáveis e específicas.
- Compare plataformas entre si quando relevante.
"""
```

5.2 System Prompt — Blog e SEO

```
SYSTEM_PROMPT_SEO = """
Você é um analista de SEO e conteúdo especializado em
marketing educacional brasileiro para concursos públicos.

Estrutura do relatório:

1. RESUMO DO BLOG
   - Sessões totais, usuários, pageviews
   - Canal principal de aquisição
   - Taxa de conversão geral

2. TOP 10 PÁGINAS (por tráfego orgânico)
   - Cliques, impressões, CTR, posição média

3. QUICK WINS DE SEO
   - Queries posição 5-20 com muitas impressões
   - Ações específicas por página

4. PROBLEMAS DE CTR
   - Páginas posição <5 com CTR <5%
   - Sugestões de title tag e meta description

5. CANIBALIZAÇÃO
   - Queries com múltiplas páginas competindo
   - Sugestões de consolidação

6. OPORTUNIDADES DE CONTEÚDO
   - Temas com busca mas sem conteúdo no blog
   - Artigos desatualizados para refresh

7. COMPARAÇÃO SEMANAL
   - Variações vs semana anterior
   - Páginas que ganharam ou perderam posição

Regras:
- Nunca invente dados ou posições.
- Priorize conteúdo transacional (inscrição, edital, vagas)
  vs informacional.
- Sugestões de title/description devem usar gatilhos do
  nicho (datas, vagas, salários).
"""
```

5.3 Chamada Unificada

```
import anthropic
client = anthropic.Anthropic(api_key="SUA_API_KEY")

def gerar_relatorio_organico(dados_social, dados_blog_seo):
    # Relatório 1: Redes Sociais
```

```
    rel_social = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=4000,
        system=SYSTEM_PROMPT_SOCIAL,
        messages=[{"role": "user", "content": dados_social}]
    ).content[0].text

    # Relatório 2: Blog + SEO
    rel_seo = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=4000,
        system=SYSTEM_PROMPT_SEO,
        messages=[{"role": "user", "content": dados_blog_seo}]
    ).content[0].text

    return rel_social, rel_seo
```

5.4 Custo Estimado Total (Ads + Orgânico)

Pipeline	Frequência	Custo mensal estimado
Ads (KPIs + criativos)	Diário	~US$ 2,70
Social orgânico	Semanal + alertas diários	~US$ 1,50
Blog + SEO	Semanal	~US$ 0,40
TOTAL		~US$ 4,60/mês

6. Estratégia de Agendamento

Execução	Frequência	Conteúdo	CRON (UTC)
Alerta diário	Seg a Sex, 07h BRT	Anomalias, posts viralizando, quedas	0 10 * * 1-5
Relatório Social	Segunda, 07h BRT	Análise completa de todas as redes	0 10 * * 1
Relatório SEO	Quarta, 07h BRT	Blog + Search Console (delay 3 dias)	0 10 * * 3
Mensal consolidado	1º dia útil, 07h BRT	Social + Blog + SEO + ads (cross-analysis)	0 10 1 * *

7. Estrutura de Arquivos

```
relatorio-organico/
├── main.py
├── collectors/
│   ├── instagram.py
│   ├── facebook.py
│   ├── youtube.py
│   ├── tiktok.py
│   ├── google_analytics.py
│   └── search_console.py
├── analysis/
│   ├── claude_client.py
│   ├── prompts_social.py
│   └── prompts_seo.py
├── delivery/
│   ├── email_sender.py
│   ├── slack_sender.py
│   └── whatsapp_sender.py
├── config/
│   ├── settings.py
│   └── service-account.json
├── data/historico/
├── logs/
├── requirements.txt
├── .env
└── .github/workflows/
    ├── daily_alerts.yml
    ├── weekly_social.yml
    └── weekly_seo.yml
```

8. Credenciais Necessárias

Serviço	O que precisa	Onde conseguir	Observação
Claude API	API Key	console.anthropic.com	Mesma do pipeline de ads
IG + Facebook	System User Token	Business Manager	Mesmo token do pipeline de ads
YouTube	OAuth2 credentials	Google Cloud Console	Habilitar Data API v3 + Analytics
TikTok	TikAPI key	tikapi.io	Opcional na fase 1
GA4 + GSC	Service Account JSON	Google Cloud > IAM	Mesmo SA para ambos

Dica: GA4 e Search Console usam o mesmo Service Account. O token do Business Manager (Meta) é o mesmo do pipeline de ads — basta garantir permissões de insights para páginas e contas IG.

9. Dependências (requirements.txt)

```
anthropic>=0.40.0
google-api-python-client>=2.130.0
google-analytics-data>=0.18.0
google-auth>=2.29.0
google-auth-oauthlib>=1.2.0
requests>=2.31.0
python-dotenv>=1.0.0
pandas>=2.2.0
sendgrid>=6.11.0
twilio>=9.0.0
# tikapi>=0.3.0  # descomentar se usar TikAPI
```

10. Integração com Pipeline de Ads

Os dois pipelines podem compartilhar módulos: delivery, config e claude_client. No relatório mensal consolidado, o Claude recebe dados dos dois pipelines juntos e gera uma análise cruzada.

Exemplos de insights cruzados: correlação entre pico de tráfego orgânico no blog e queda de CPA nos ads (conteúdo educativo aquecendo o funil); posts orgânicos com alto engajamento que podem virar criativos de ads; queries do Search Console que revelam demanda para campanhas de Search no Google Ads; temas de Reels com alta retenção que indicam dores do público para explorar em LPs.

11. Evoluções Futuras

1. **Competição orgânica:** Integrar Semrush ou Ahrefs API para monitorar concorrentes (Gran Cursos, Alfacon, Estrategia).
2. **SEO automático:** Claude sugere title tags e meta descriptions; script aplica via WordPress REST API.
3. **Calendário editorial:** Claude analisa dados e propõe pauta semanal baseada em queries em alta, temas com engajamento e datas de editais.
4. **Análise de thumbnails:** Enviar thumbnails do YouTube/TikTok via vision para avaliar qualidade visual + CTR.
5. **Monitoração de índice:** Alerta se páginas importantes forem desindexadas (Search Console + Indexing API).